

HATEOAS

- HATEOAS (Hypermedia As The Engine Of Application State) .
- In Spring Boot it enables the creation of RESTful APIs where clients interact with the application through hypermedia links embedded in API responses, rather than relying on hardcoded URLs.
- This approach promotes discoverability and reduces coupling between the client and server.

- HATEOAS stands for Hypermedia as the Engine of Application State. It is a key constraint of REST that makes APIs self-explanatory by including links inside responses.
- Without HATEOAS: The client needs hardcoded endpoints to know how to interact with the API.
- With HATEOAS: The API guides the client on what actions are available dynamically

Example: Traditional REST API (Without HATEOAS)

```
{  
  "id": 1,  
  "name": "John Doe",  
  "email": "john@example.com"  
}
```

Example: HATEOAS-Driven API (With Links)

```
{  
  "id": 1,  
  "name": "John Doe",  
  "email": "john@example.com",  
  "links": [  
    {"rel": "self", "href": "/users/1"},  
    {"rel": "update", "href": "/users/1/update"},  
    {"rel": "delete", "href": "/users/1/delete"}  
  ]  
}
```

Key Concepts:

- **Hypermedia Links:** API responses include links to related resources or actions that the client can follow. These links specify the URI, HTTP method, and the relation (e.g., "self," "next," "delete").
- **Discoverability:** Clients can discover available actions and navigate through the application state by following the provided links, eliminating the need for prior knowledge of the API's structure beyond an initial entry point.
- **Reduced Coupling:** HATEOAS minimizes the tight coupling between client and server by allowing the server to dynamically provide relevant actions, making the API more flexible and maintainable.

Dependency:

- Include the spring-boot-starter-hateoas dependency in your pom.xml or build.gradle.

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-hateoas</artifactId>  
</dependency>
```

- The API provides links (href) inside the response.
- The client doesn't need hardcoded endpoints — it follows the links dynamically.
- The client knows what actions are possible (self, update, delete).
-  Why Use HATEOAS in REST APIs?
 -  Self-Descriptive API Responses — The API tells clients what actions are available.
 -  Reduces Hardcoding — Clients don't need to memorize API URLs.
 -  Improves API...

Link Creation

- WebMvcLinkBuilder simplifies the creation of links by avoiding hardcoded URIs and leveraging Spring MVC controller methods.

```
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;
```

```
    Link selfLink =  
    linkTo(methodOn(MyController.class).getResource(id)).withSelfRel();
```

```
    Link relatedLink =  
    linkTo(methodOn(MyController.class).getRelatedResource(id)).withRel("related");
```

Without hateos

```
@GetMapping("/customer")
```

```
public List<Customer> getAll() {  
    return customerService.getAll();  
}
```

No link will be generated

- With hateos

```
public CollectionModel<EntityModel<Customer>>  
getAll() {
```

```
    List<EntityModel<com.cts.model.Customer>>  
    customers = customerService.getAll().stream()  
        .map(customer -> EntityModel.of(customer,
```

```
            linkTo(methodOn(CustomerController.class).searchCu  
stomerById(customer.getId()))).withSelfRel()
```

```
        ))
```

```
        .collect(Collectors.toList());
```

```
    return CollectionModel.of(customers,  
        linkTo(methodOn(CustomerController.class).getAll()).wi  
thSelfRel());  
}
```



```
@GetMapping("/customer/{id}")
public Customer
searchCustomerById(@PathVariable Integer
id) {
    return
customerService.searchCustomerById(id);
}
```

```
@GetMapping("/customer/{id}")
public EntityModel<Customer>
searchCustomerById(@PathVariable Integer id) {
    var customer =
customerService.searchCustomerById(id);
    return EntityModel.of(customer,
linkTo(methodOn(CustomerController.class).searchCustomerById(id)).withSelfRel(),

linkTo(methodOn(CustomerController.class).getAll()).withRel("customers"));

}
```

output

<http://localhost:8080/customer>

```
{
  "_embedded": {
    "customerList": [
      {
        "id": 1,
        "name": "string",
        "salary": 0.1,
        "dob": "2025-10-14",
        "address": {
          "addrId": 1,
          "street": "string",
          "city": "string",
          "state": "string"
        },
        "carSet": [],
        "_links": {
          "self": {
            "href": "http://localhost:8080/customer/1"
          }
        }
      }
    ]
  },
  "_links": {
    "self": {
      "href": "http://localhost:8080/customer"
    }
  }
}
```

<http://localhost:8080/customer/1>

```
{
  "id": 1,
  "name": "string",
  "salary": 0.1,
  "dob": "2025-10-14",
  "address": {
    "addrId": 1,
    "street": "string",
    "city": "string",
    "state": "string"
  },
  "carSet": [],
  "_links": {
    "self": {
      "href": "http://localhost:8080/customer/1"
    }
  },
  "customers": {
    "href": "http://localhost:8080/customer"
  }
}
```

Benefits:

- Improved API Design: Promotes a more robust and self-documenting API.
- Enhanced Client-Side Flexibility: Clients can adapt to API changes more easily without requiring code modifications.
- Better Maintainability: Reduces the impact of changes to resource URIs on client applications.